Data Architectures

What Is Data Architecture?

Data Architecture is the blueprint that defines **how data is collected, stored, processed, managed, and consumed** within a system.

Think of data architecture as:

- The **building plan** of your data system
- The **logic** behind where data lives and how it flows
- The **connection** between raw data and meaningful insights

Key Idea

Data architecture answers these questions:

- Where does the data come from?
- Where is it stored?
- How is it processed?
- How is it accessed by applications or users?
- How is data kept secure, scalable, and reliable?

If your FYP uses data, then it already has a data architecture — even if you didn't design it consciously.

Importance

For student projects, data architecture helps you:

- Avoid messy and unstructured data handling
- Scale your project beyond demo-level
- Explain your system clearly in documentation and viva
- Align your project with **industry-level practices**

Without Data Architecture

- Data scattered everywhere
- Hard-coded logic
- No scalability
- Difficult to explain to supervisors

With Proper Data Architecture

- Clean flow of data
- Modular design

- Easier debugging and extension
- Professional-grade project

Core Components of Data Architecture

Every data architecture (simple or complex) consists of these core components:

1. **Data Sources**
    - User input
    - Sensors
    - APIs
    - Files (CSV, JSON, logs)
    - Databases
2. **Data Ingestion Layer**
    - Batch ingestion
    - Real-time ingestion
    - APIs, pipelines, message queues
3. **Data Storage**
    - Relational databases (SQL)
    - NoSQL databases
    - Data lakes
    - Data warehouses
4. **Data Processing Layer**
    - ETL / ELT
    - Stream processing
    - Machine learning pipelines
5. **Data Serving / Consumption Layer**
    - Dashboards
    - APIs
    - ML models
    - Applications
6. **Governance & Security (Optional but Important)**
    - Access control
    - Data validation

Types of Data Architectures

Below are the most common data architectures, ranging from **simple to advanced**. These are highly relevant for FYPs.

Traditional (Monolithic) Data Architecture

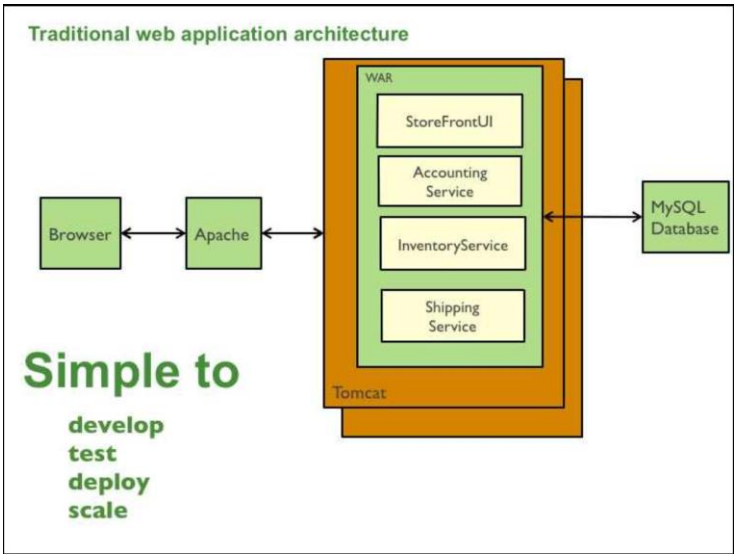
Description

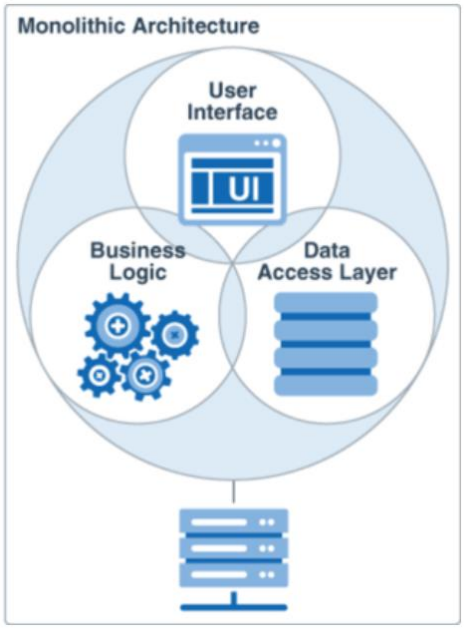
In this architecture, **all components are tightly coupled**:

- Data storage
- Business logic
- Data processing
- User interface

Everything runs as a **single system**.

Diagram





Suitable FYP Examples

- Simple CRUD web apps
- Library management system
- Student record system

When to Use

- Small projects
- No real-time or big data
- Single database

Layered Data Architecture

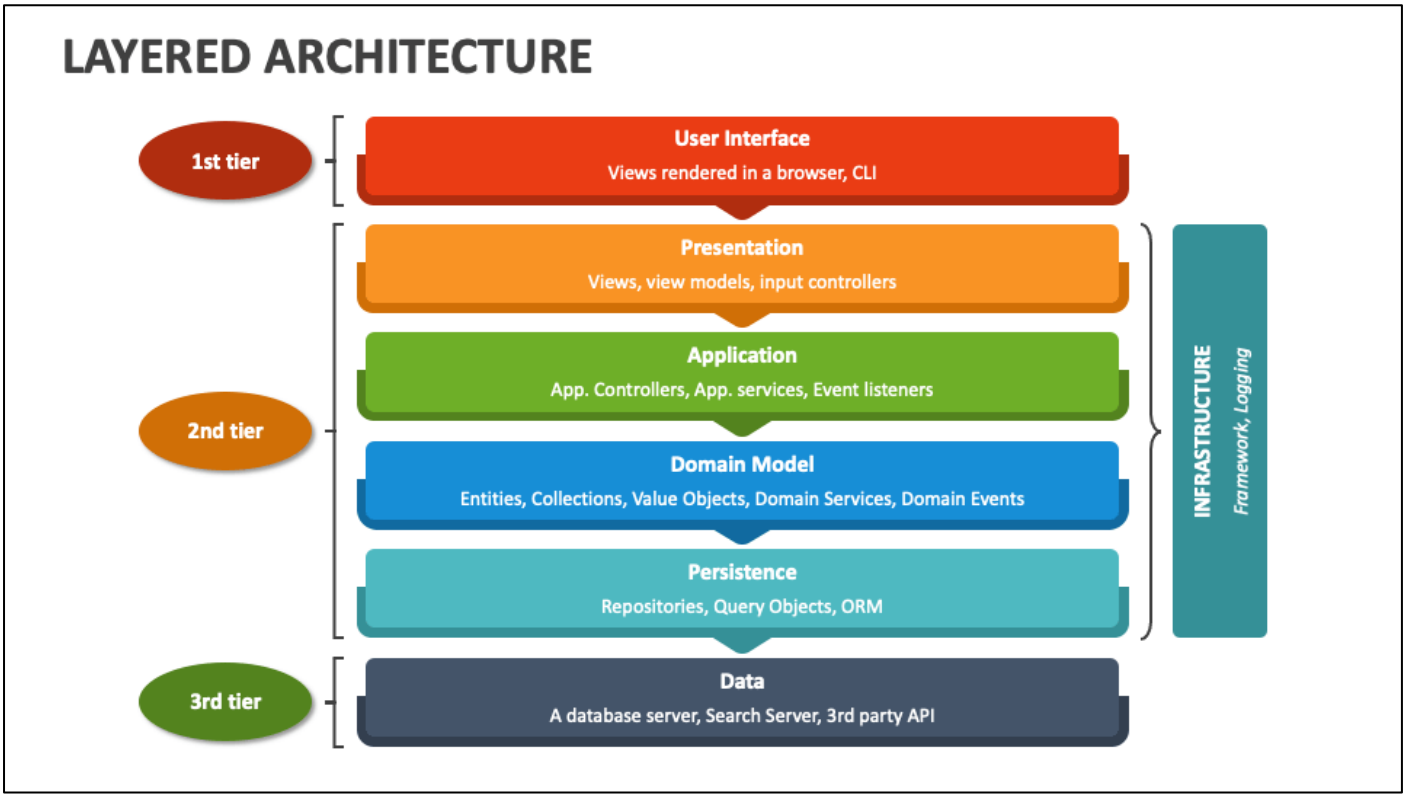
Description

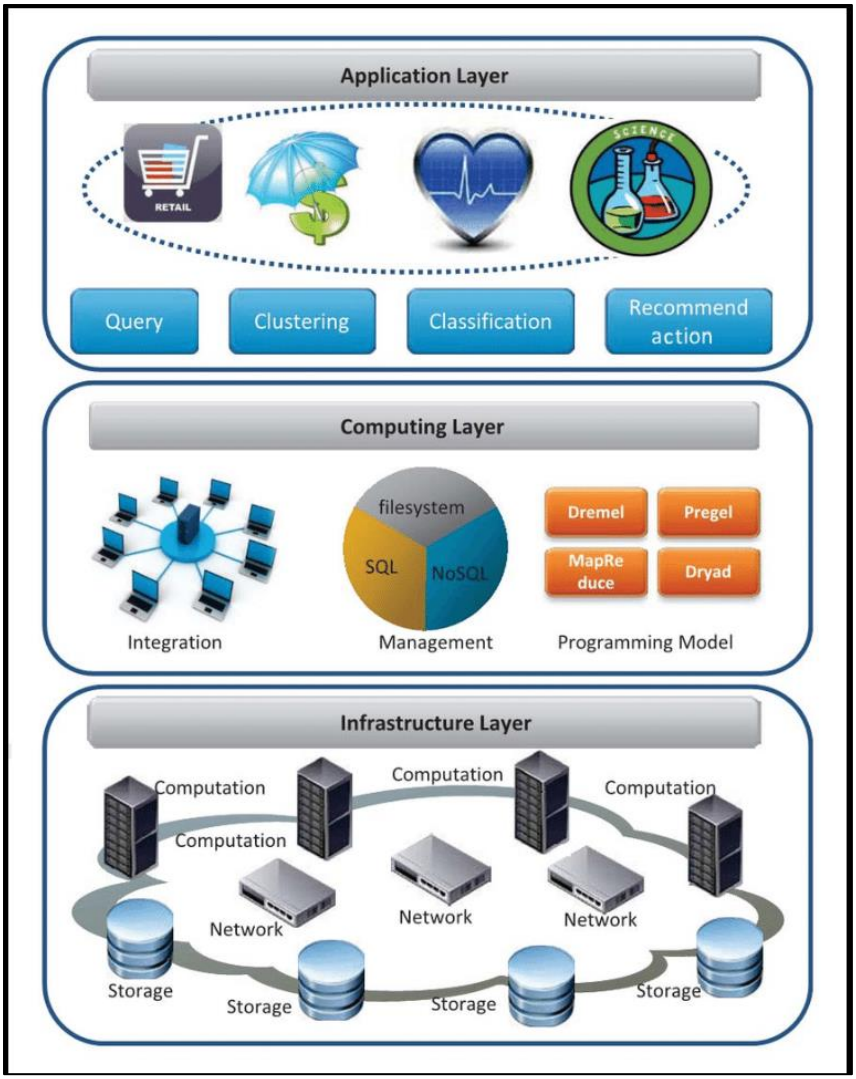
Data flows through **clearly defined layers**, each with a specific responsibility.

Typical layers:

1. Presentation Layer
2. Application Layer
3. Data Layer

Diagram





Suitable FYP Examples

- Web applications
- ML-backed dashboards
- REST API projects

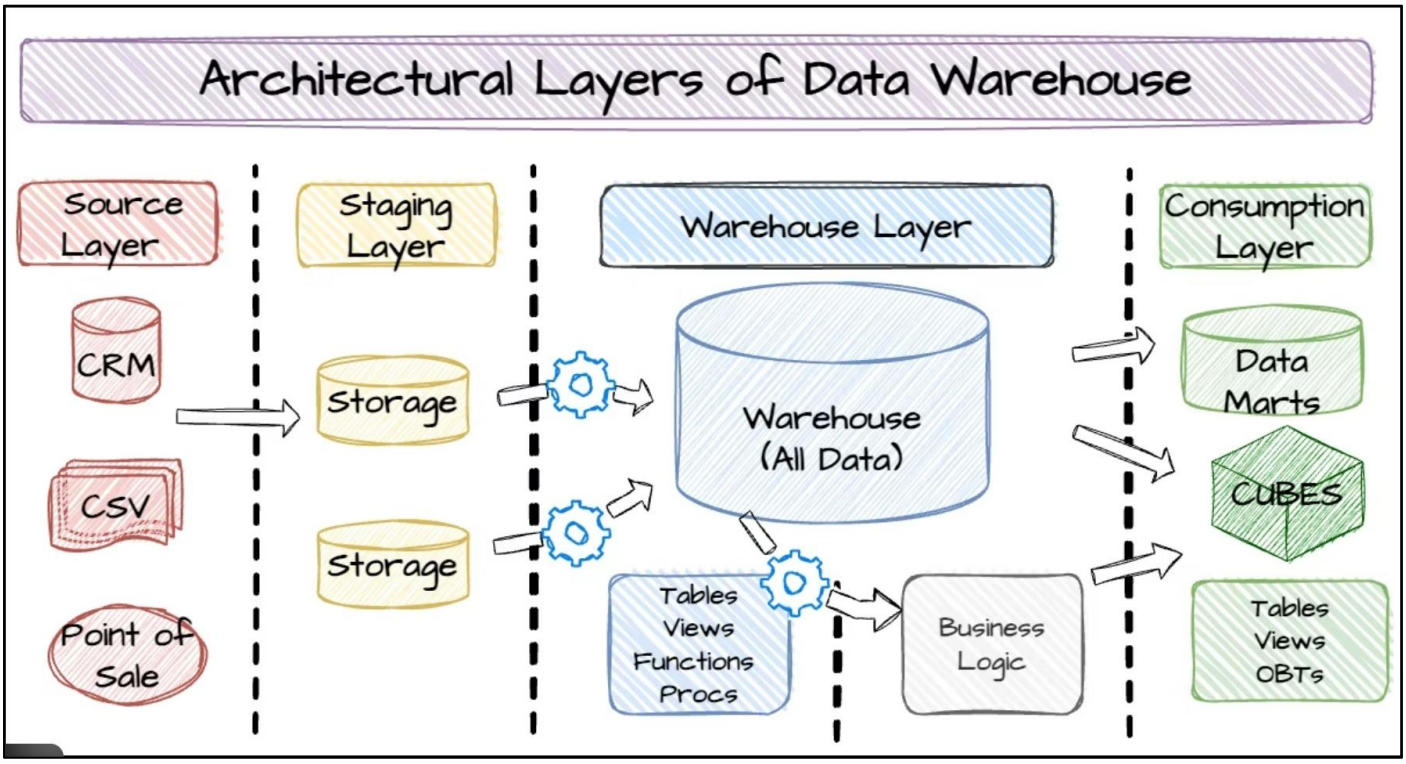
Data Warehouse Architecture

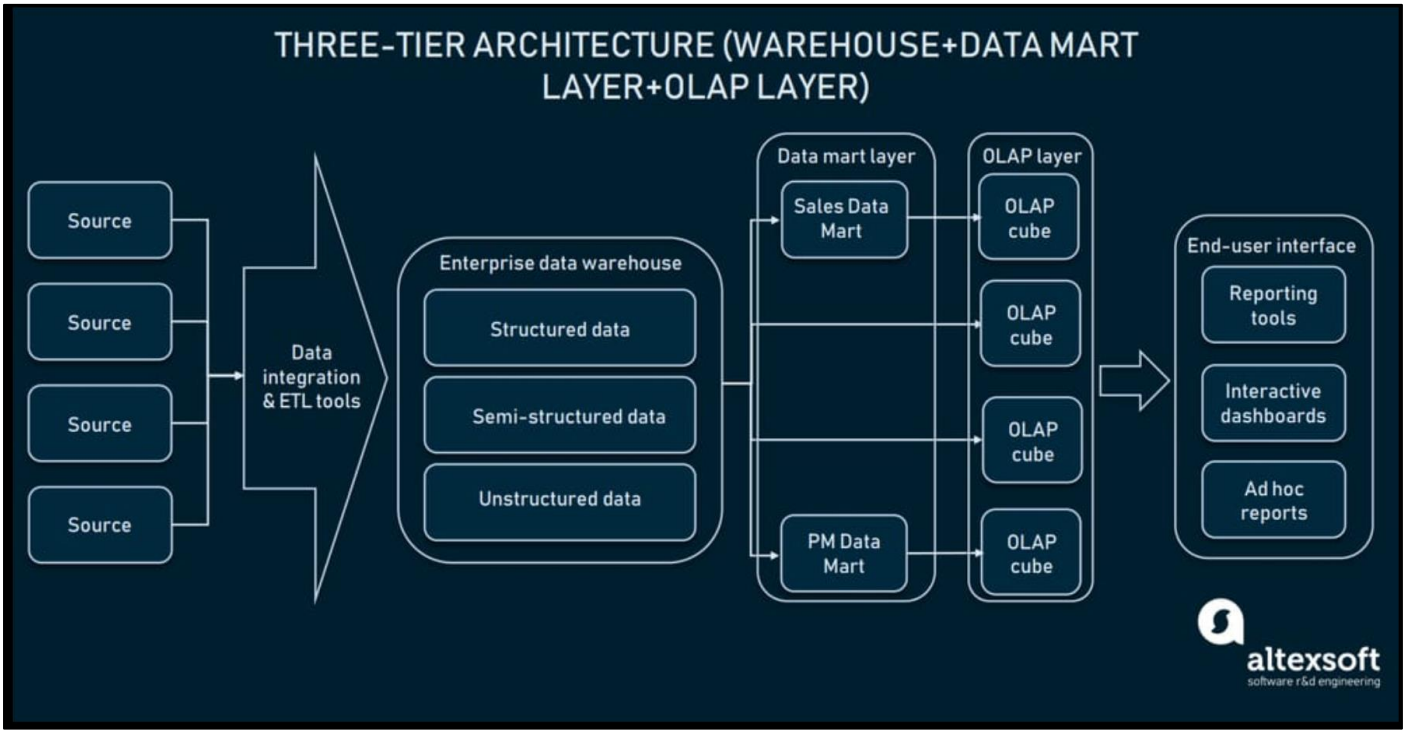
Designed for **analytics and reporting**, not transactions.

Data is:

- Collected from multiple sources
- Cleaned and transformed
- Stored in a structured format

Diagram





Suitable FYP Examples

- Business intelligence systems
- Sales analysis projects
- University analytics systems

Data Lake Architecture

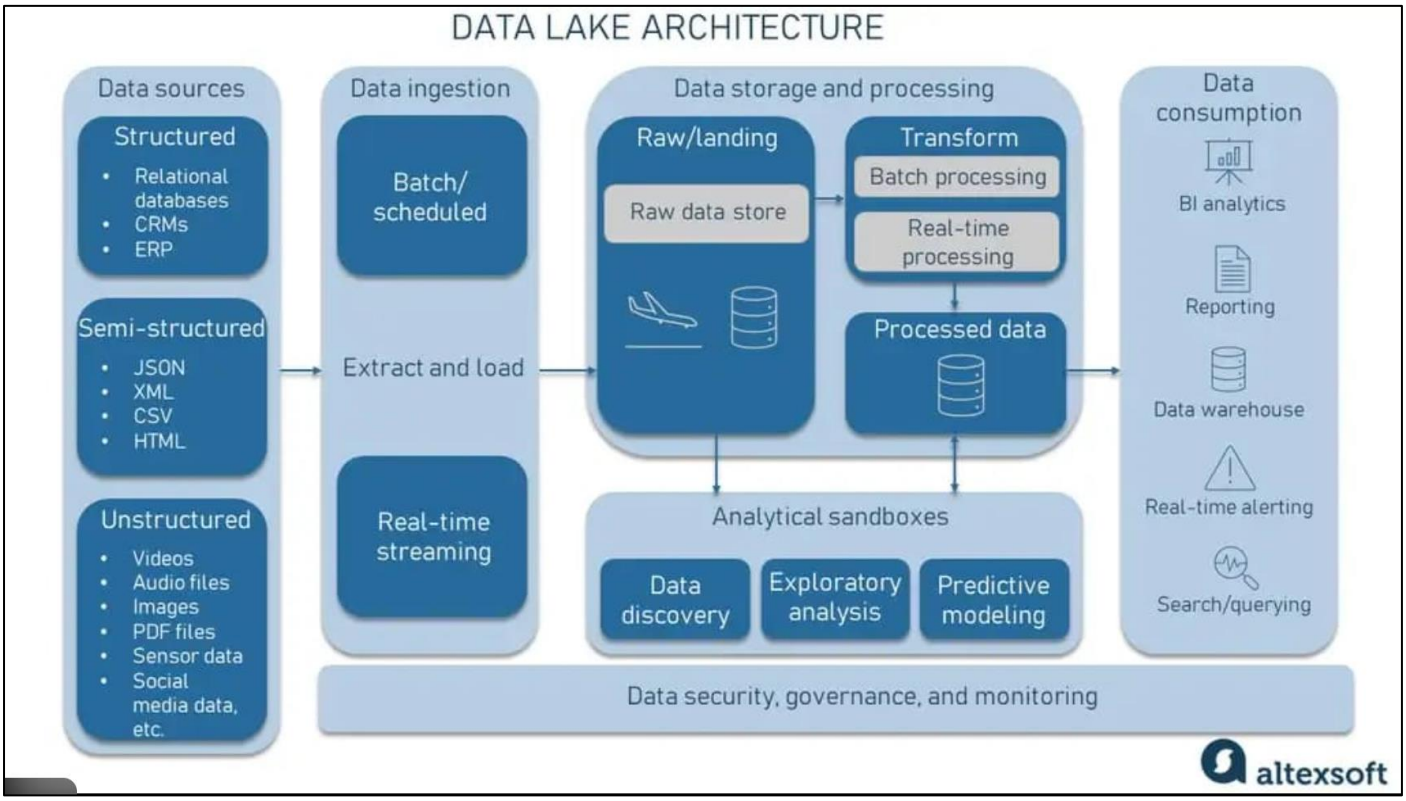
Description

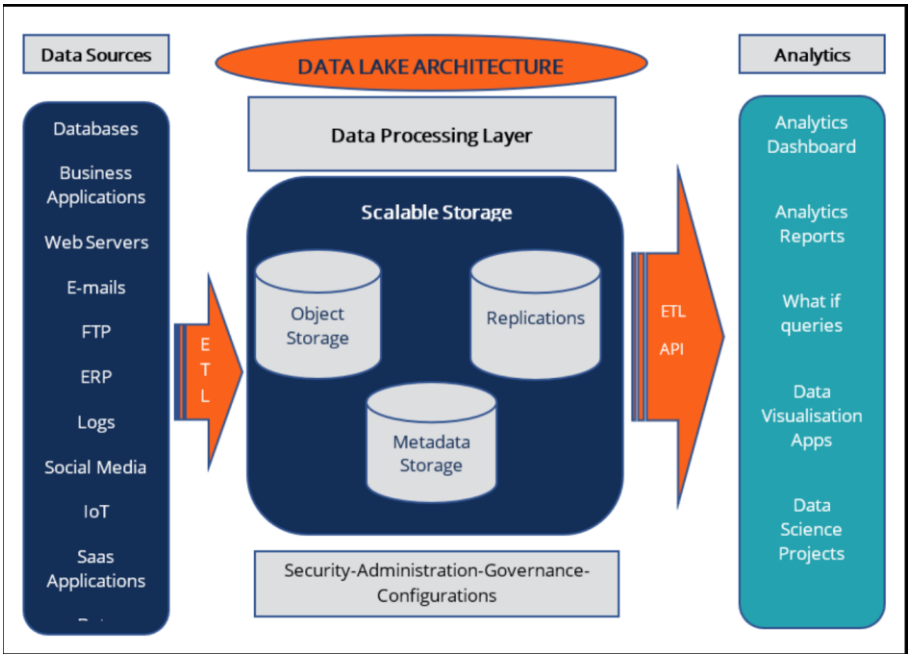
Stores **raw data in its original format**.

Supports:

- Structured data
- Semi-structured data
- Unstructured data

Diagram





Suitable FYP Examples

- Big data projects
- ML / AI systems
- NLP and CV projects

Lambda Architecture

Lambda Architecture is a data architecture designed to handle large volumes of data by processing it in two parallel paths:

1. **Batch Processing (accurate but slow)**
2. **Real-Time / Speed Processing (fast but approximate)**

The results of both paths are combined to provide **low-latency and highly accurate outputs**.

This architecture is especially useful when systems need:

- Real-time insights
- Historical accuracy
- Fault tolerance

Core Idea

Imagine you are calculating student performance:

- You want **instant results** when new data arrives (speed layer)
- You also want **correct and complete results** after processing all historical data (batch layer)

Lambda Architecture allows **both** to happen at the same time.

Main Layers of Lambda Architecture

Lambda Architecture consists of **three main layers**:

1. Batch Layer

- Stores **master data set** (immutable, append-only)
- Processes **all historical data**
- Produces accurate views

Characteristics:

- High latency
- High accuracy
- Heavy computation

Examples:

- Hadoop
- Spark (batch mode)

2. Speed Layer (Real-Time Layer)

- Processes **new incoming data instantly**
- Compensates for batch layer latency

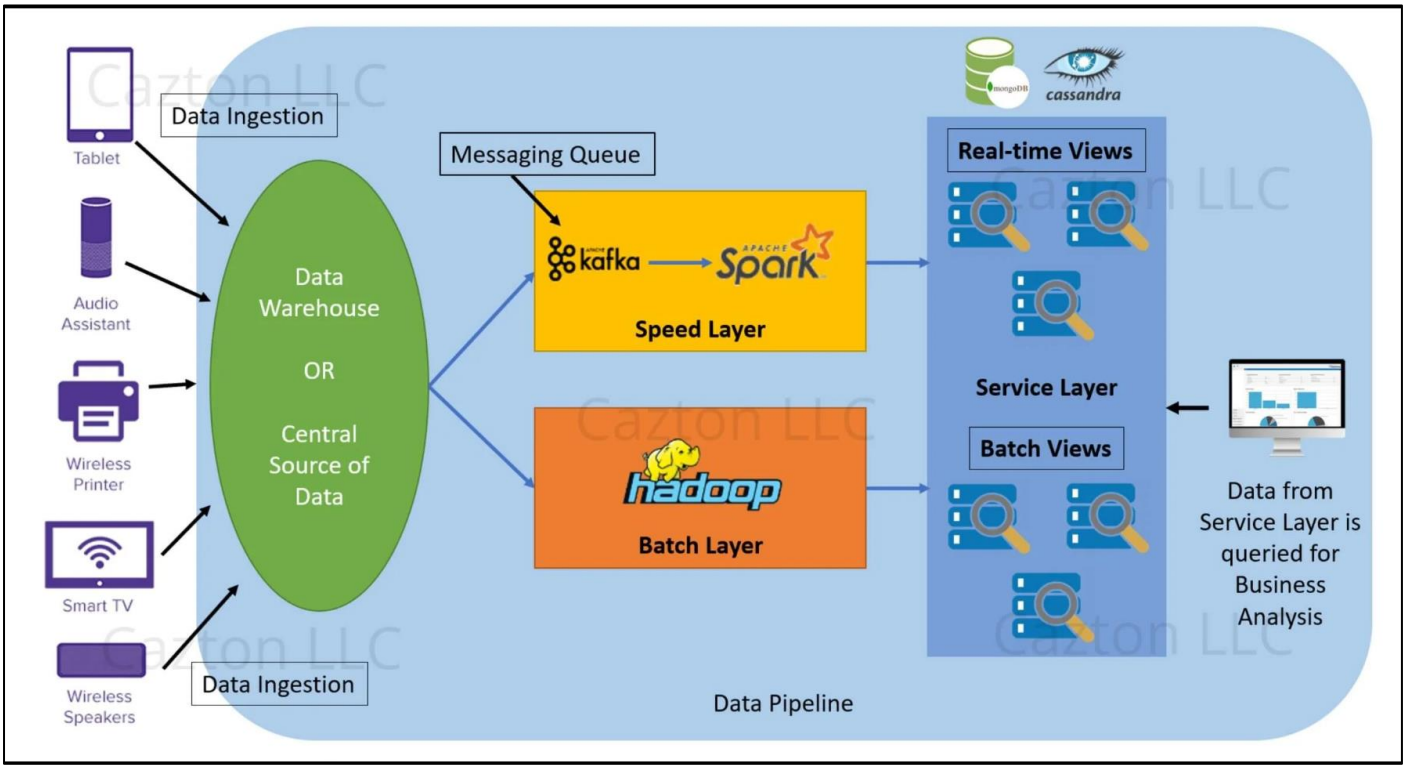
3. Serving Layer

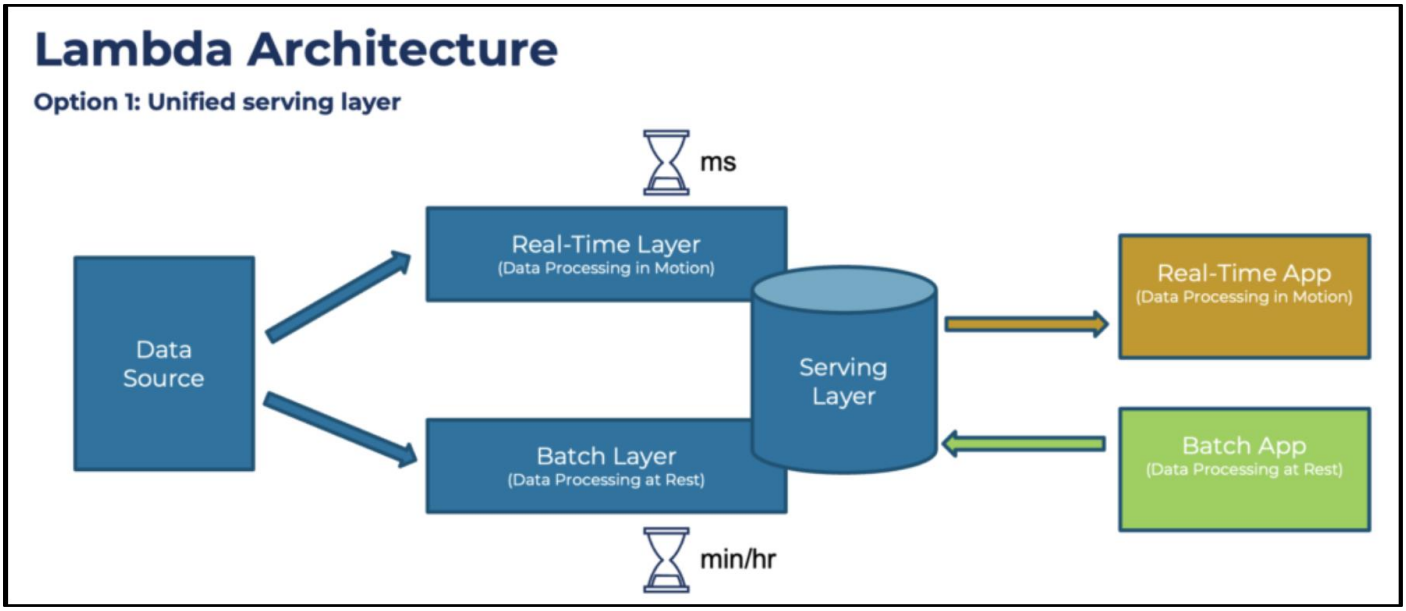
- Merges outputs from batch and speed layers
- Provides query able views

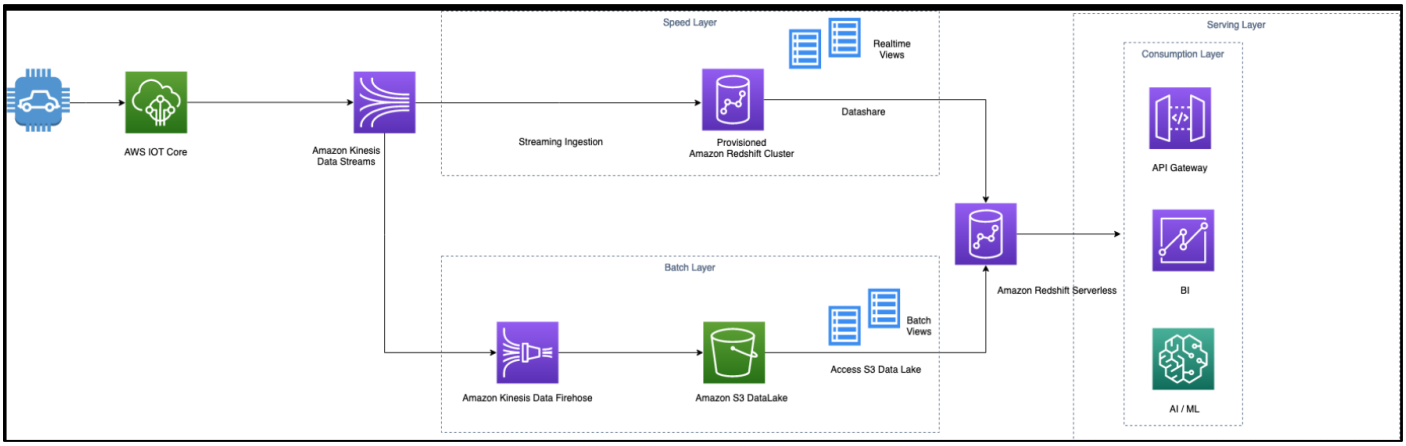
Lambda Architecture Data Flow

1. Data is generated from sources (users, sensors, logs, APIs)
2. Data is sent to **both batch and speed layers simultaneously**
3. Batch layer processes full historical data periodically
4. Speed layer processes recent data in real-time
5. Serving layer combines both results
6. Final output is delivered to applications

Diagram







Kappa Architecture

Kappa Architecture is a simplified data architecture designed to handle real-time streaming data using a single processing path.

Unlike Lambda Architecture, Kappa Architecture **eliminates the batch layer** and relies entirely on **stream processing**, making the system simpler and easier to maintain.

This architecture is ideal when:

- Data is continuously generated
- Real-time processing is the main requirement
- Reprocessing can be done by replaying streams

Core Idea

Think of Kappa Architecture as:

- "Do everything in real time"
- "Store data as streams, not static files"

Instead of having:

- One system for old data (batch)

- Another system for new data (speed)

Kappa Architecture uses **one streaming pipeline** for both.

Main Components of Kappa Architecture

Kappa Architecture has **three core components**:

1. Streaming Data Source

- Data is treated as a **continuous stream**
- All events are stored in an append-only log

Examples:

- Apache Kafka
- Amazon Kinesis

2. Stream Processing Layer

- Processes data **in real time**
- Same logic is used for:
    - New data
    - Reprocessing old data (by replaying streams)

Examples:

- Apache Flink
- Kafka Streams
- Spark Structured Streaming

3. Serving / Storage Layer

- Stores processed results
- Used by applications, dashboards, and APIs

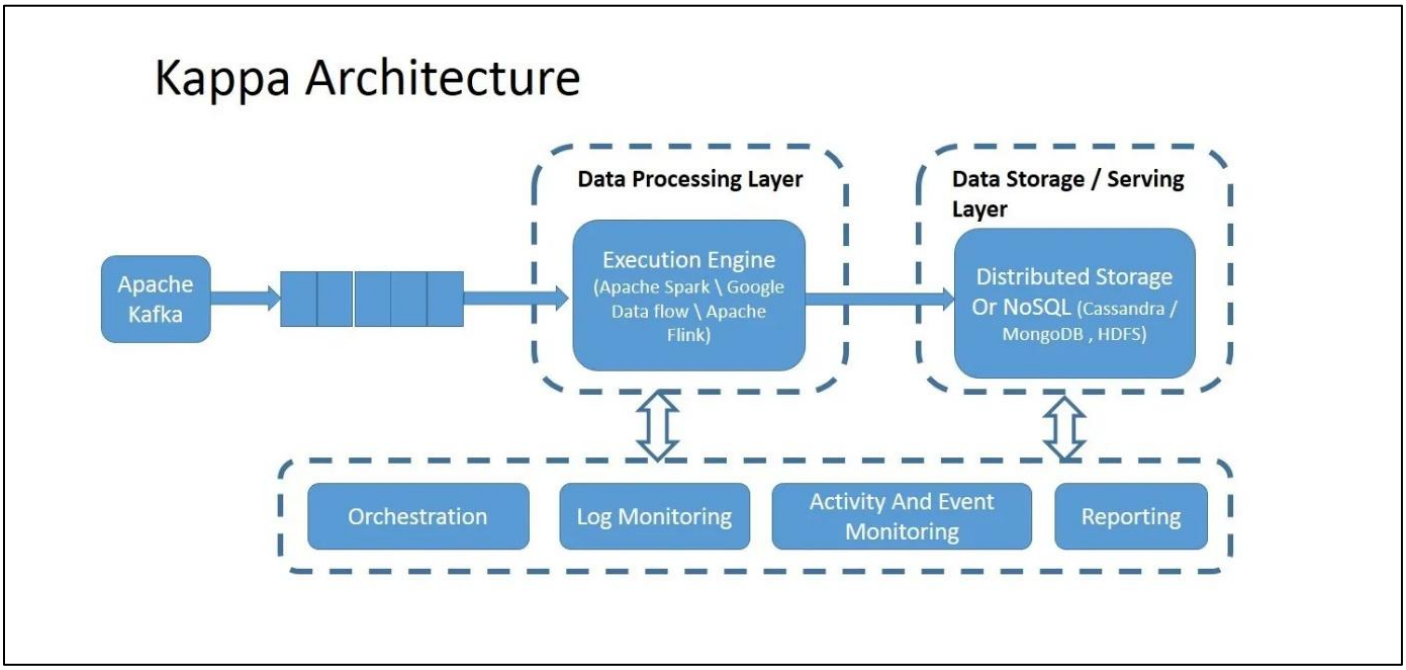
Examples:

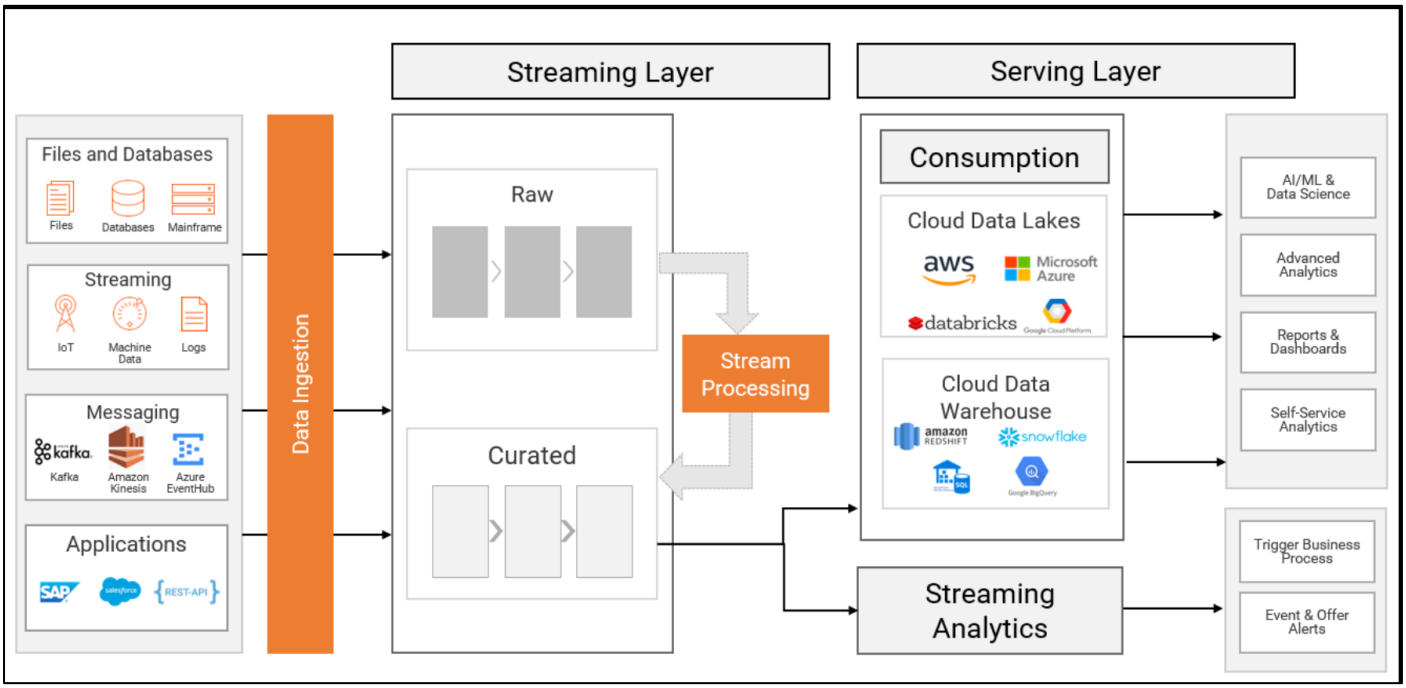
- NoSQL databases
- Elasticsearch
- Cassandra

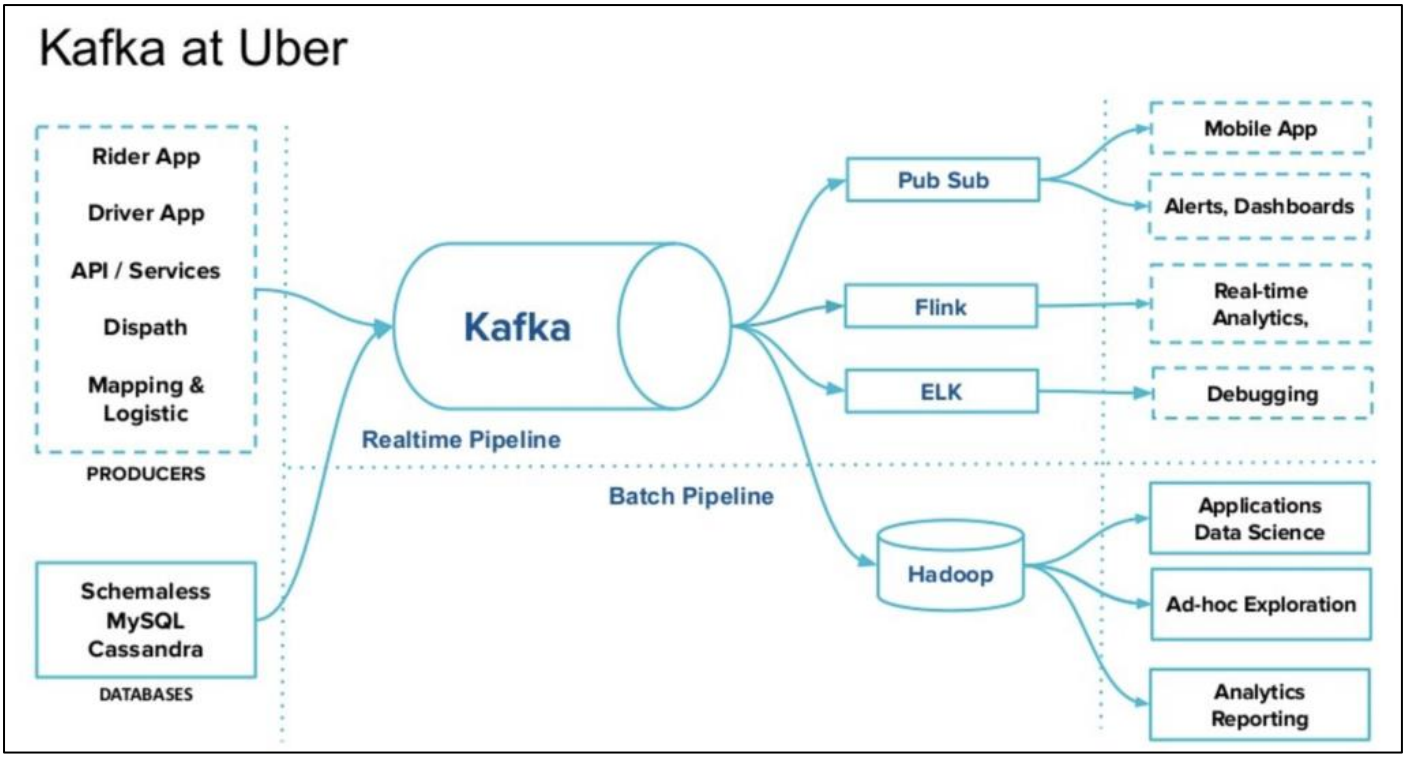
Kappa Architecture Data Flow

1. Data is generated from sources (users, sensors, logs)
2. Data is written to a streaming platform (event log)
3. Stream processor consumes data in real time
4. Results are stored in the serving layer
5. Applications query the serving layer

Diagram


Kappa Architecture
Data Processing Layer
Data Storage / Serving Layer
Apache Kafka
Execution Engine (Apache Spark \ Google Data flow \ Apache Flink)
Distributed Storage Or NoSQL (Cassandra / MongoDB , HDFS)
Orchestration
Log Monitoring
Activity And Event Monitoring
Reporting


Streaming Layer
Serving Layer
Files and Databases
Files
Databases
Mainframe
Streaming
IoT
Machine Data
Logs
Messaging
Kafka
Amazon Kinesis
Azure EventHub
Applications
Data Ingestion
Raw
Curated
Stream Processing
Consumption
Cloud Data Lakes
aws
Microsoft Azure
databricks
Cloud Data Warehouse
amazon REDSHIFT
snowflake
Streaming Analytics
AI/ML & Data Science
Advanced Analytics
Reports & Dashboards
Self-Service Analytics
Trigger Business Process
Event & Offer Alerts



Microservices-Based Data Architecture

Microservices-Based Data Architecture is an architectural approach where a system is broken down into small, independent services, and each service manages its own data.

Instead of one central database, every microservice:

- Owns its data
- Has its own database or storage
- Communicates with other services via APIs or events

This architecture is widely used in **modern, scalable, and cloud-based systems**.

Core Idea

Think of the system as a **group of small applications**, not one big application.

Each microservice:

- Solves one specific problem
- It has its own data
- Can be developed, deployed, and scaled independently

Example:

- User Service → User Database
- Orders Service → Orders Database
- Analytics Service → Analytics Database

Main Components

1. Microservices

- Independent backend services
- Each focused on a single business capability

2. Data Storage per Service

- Each service has its **own database**

3. Communication Layer

- Services communicate using:
    - REST APIs
    - gRPC
    - Event streaming

4. API Gateway (Optional but Common)

- Single entry point for clients
- Routes requests to appropriate services

Microservices Data Flow

1. Client sends request to API Gateway
2. API Gateway forwards request to relevant service
3. Service processes logic and accesses its own database
4. Response is returned to client
5. Events may be published for other services

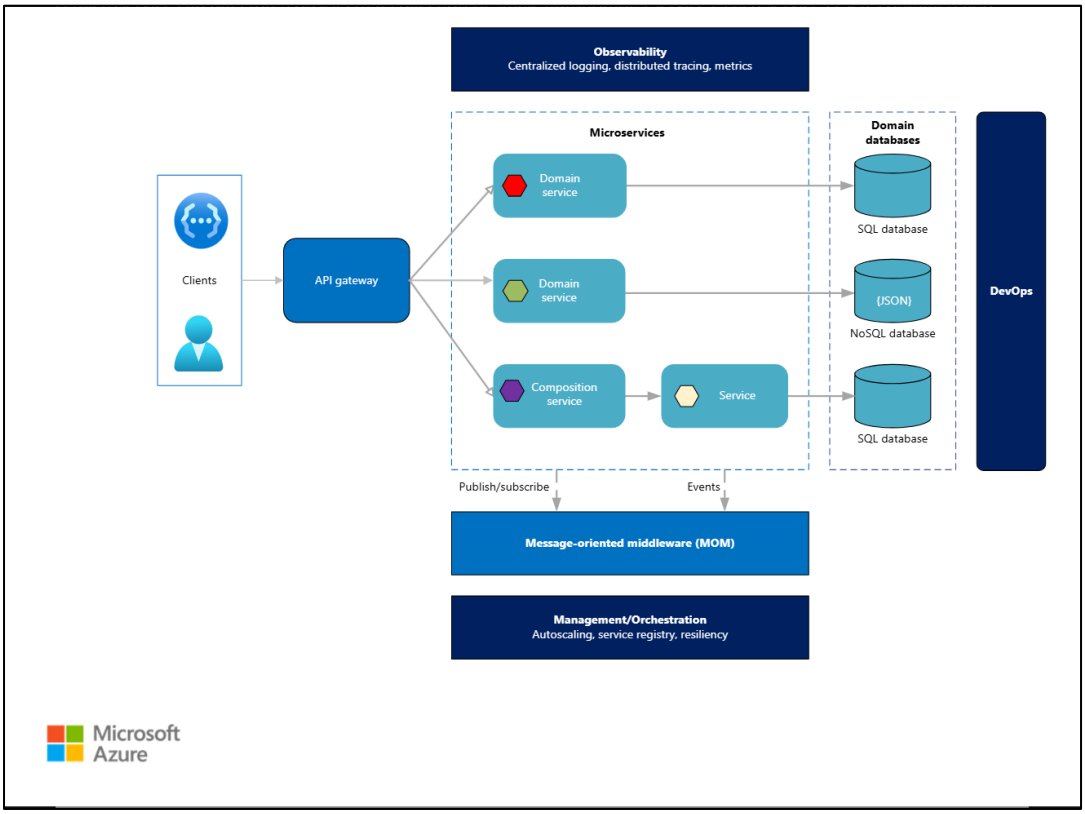
When NOT to Use Microservices Architecture

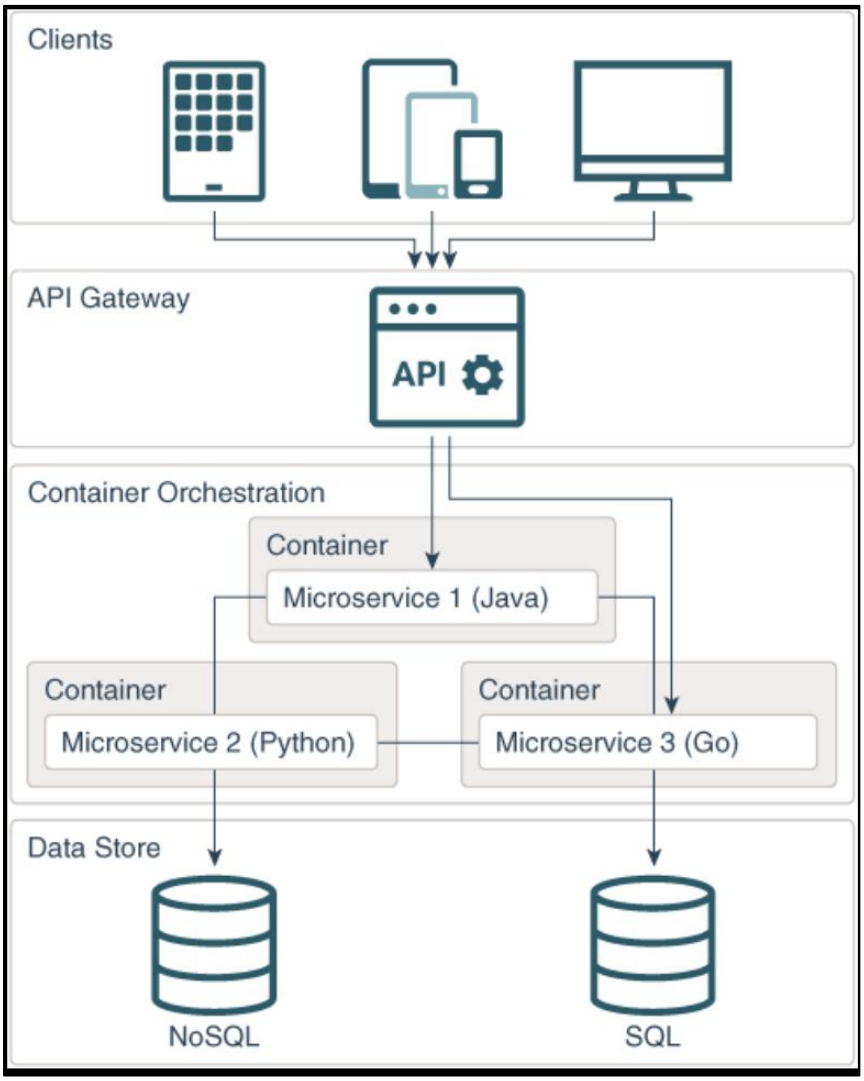
Do NOT use it if:

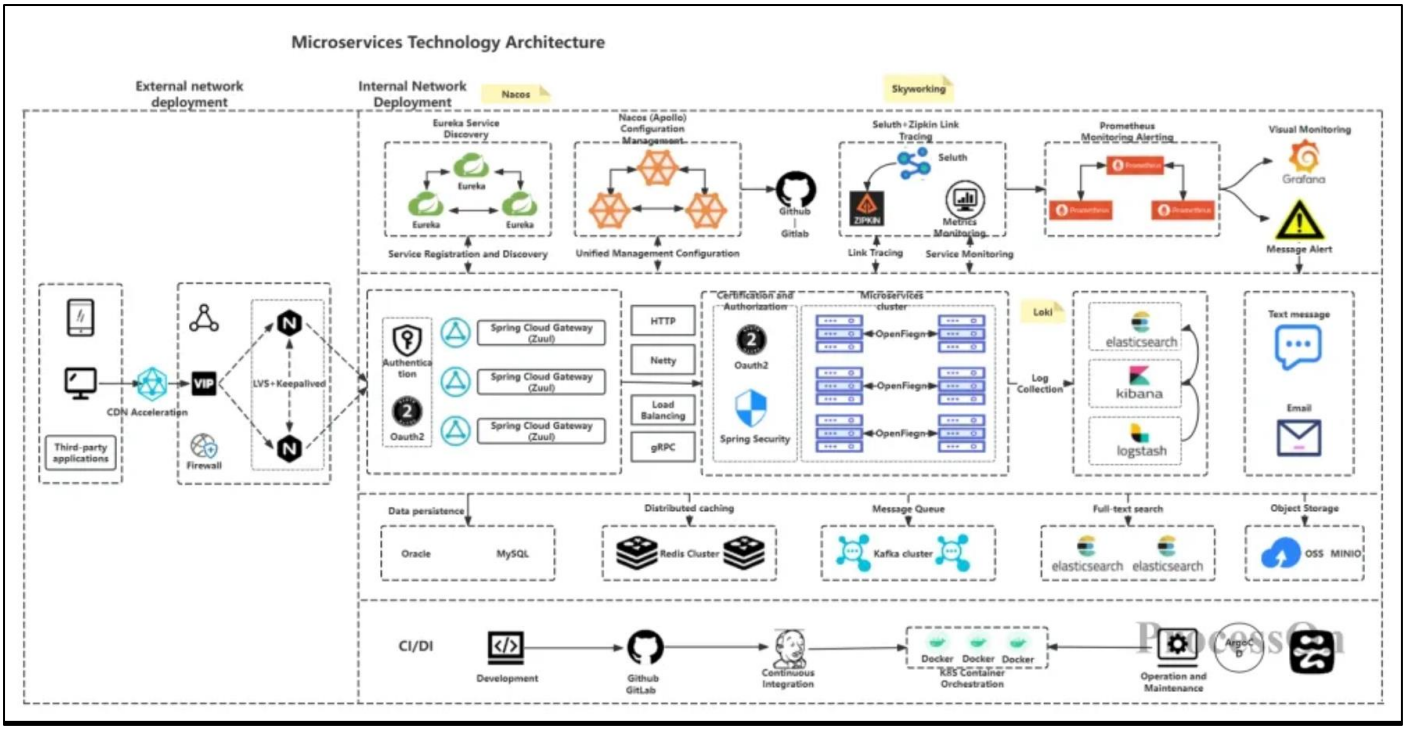
- Your FYP is small
- Team size is very limited
- You want quick development

In such cases, **Layered or Monolithic Architecture** is better.

Diagram


Observability
Centralized logging, distributed tracing, metrics
Microservices
Domain databases
Clients
API gateway
Domain service
Domain service
Composition service
Service
SQL database
{JSON}
NoSQL database
SQL database
DevOps
Publish/subscribe
Events
Message-oriented middleware (MOM)
Management/Orchestration
Autoscaling, service registry, resiliency
Microsoft Azure


Clients
API Gateway
API
Container Orchestration
Container
Microservice 1 (Java)
Container
Microservice 2 (Python)
Container
Microservice 3 (Go)
Data Store
NoSQL
SQL



Modern Cloud-Native Data Architecture

Definition

Modern Cloud-Native Data Architecture is a data architecture approach where data systems are designed, built, and deployed specifically for cloud environments using managed services, elastic scalability, and on-demand resources.

Instead of manually managing servers and infrastructure, cloud-native architecture leverages cloud platforms to automatically handle **scaling, availability, and reliability**.

Core Concept

The main idea behind cloud-native data architecture is:

- Build systems that **scale automatically**
- Use **managed cloud services** instead of self-hosted tools
- Pay only for the resources that are used
- Design systems that are **resilient and fault-tolerant by default**

In this architecture, developers focus on **data logic and analytics**, while the cloud provider manages the infrastructure.

Core Components

1. **Data Sources**
    - Web and mobile applications
    - IoT devices
    - External APIs
    - Logs and events

2. **Cloud Data Ingestion Layer**
   Responsible for ingesting data in batch or real-time.
   Examples: AWS Kinesis, Azure Event Hubs, Google Pub/Sub

3. **Cloud Storage Layer**
   Stores raw and processed data using scalable object storage.
   Examples: Amazon S3, Azure Data Lake Storage, Google Cloud Storage

4. **Data Processing & Analytics Layer**
   Performs ETL, analytics, and ML processing using managed or serverless compute.
   Examples: AWS Glue, Google Dataflow, Azure Synapse

5. **Data Serving & Consumption Layer**
   Delivers data to dashboards, APIs, and machine learning models.
   Examples: Big Query, Redshift, Snowflake

Data Flow (Step-by-Step)

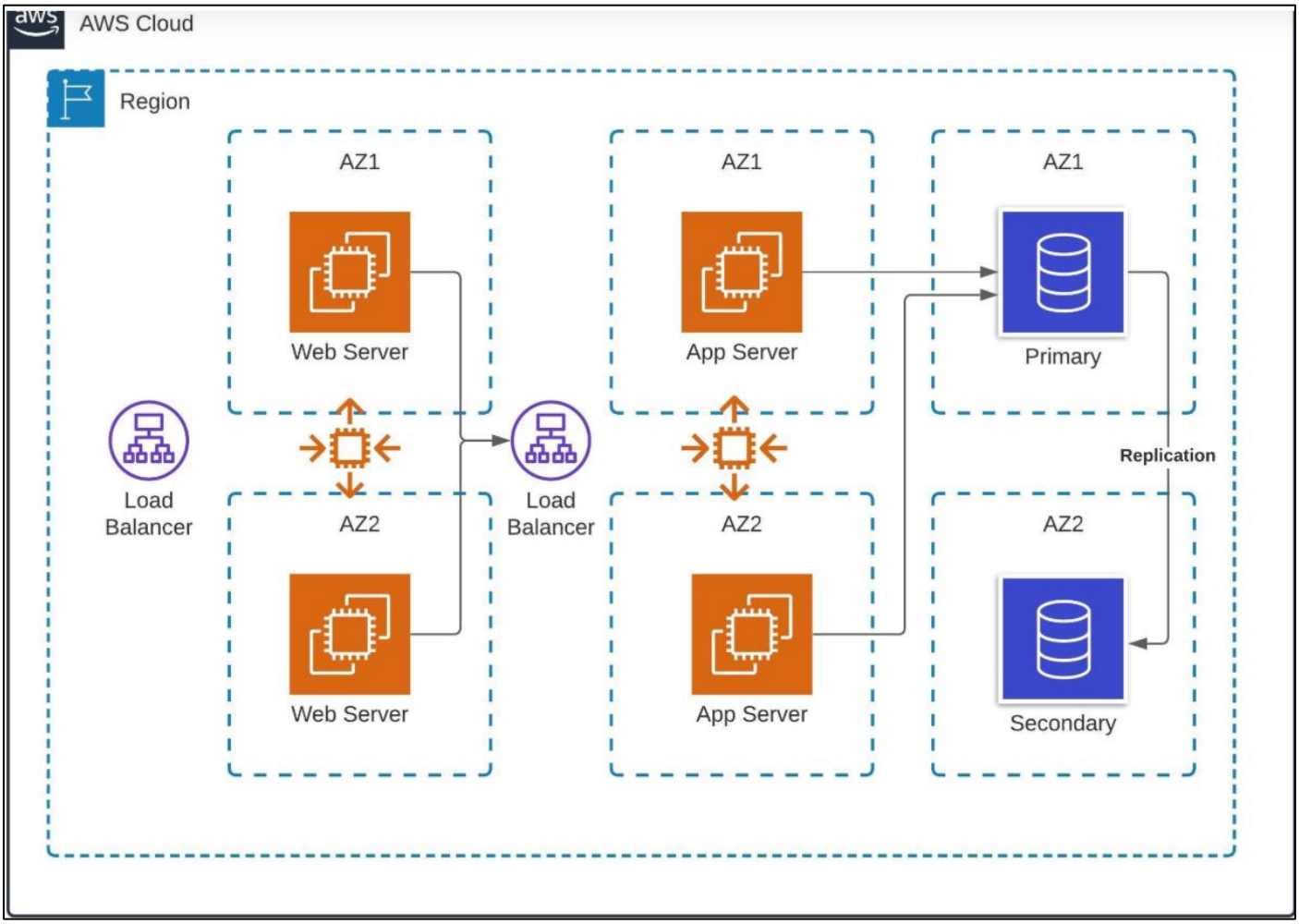
1. Data is generated by applications, users, or devices
2. Data is ingested using cloud-native ingestion services
3. Raw data is stored in cloud storage (data lake)
4. Data is processed using managed analytics or serverless services
5. Processed data is served to applications, dashboards, or ML models

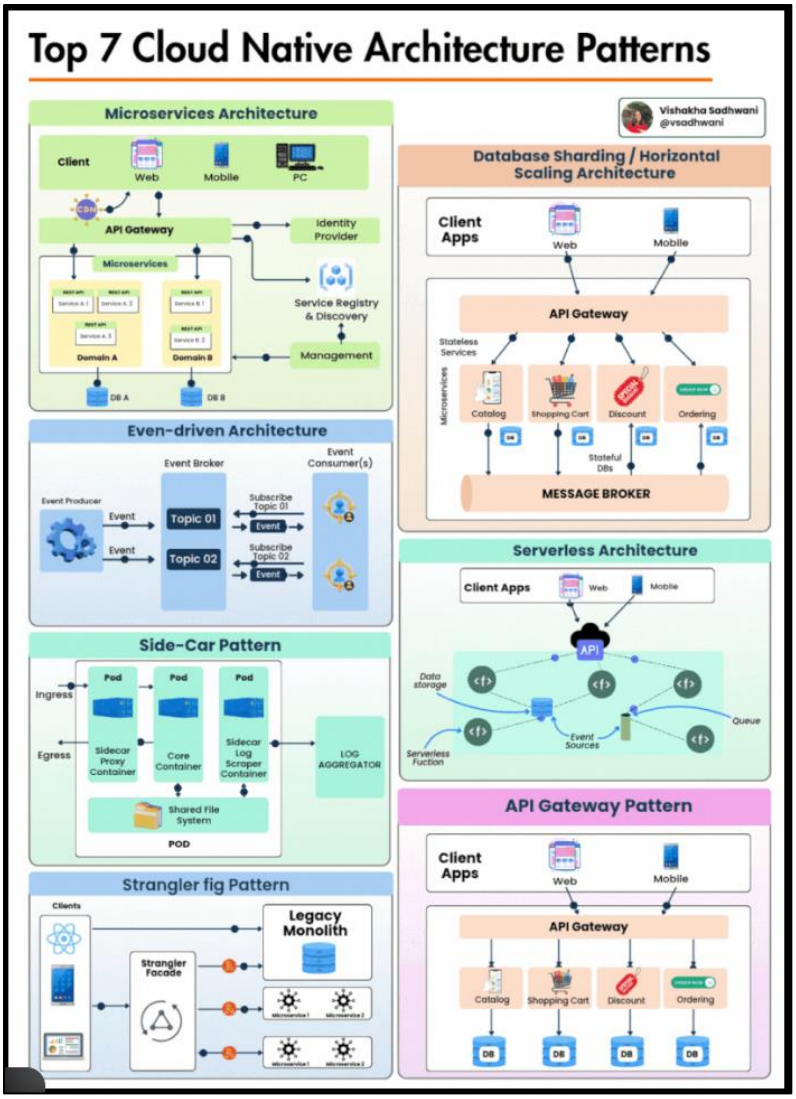
When to Use Cloud-Native Data Architecture in an FYP

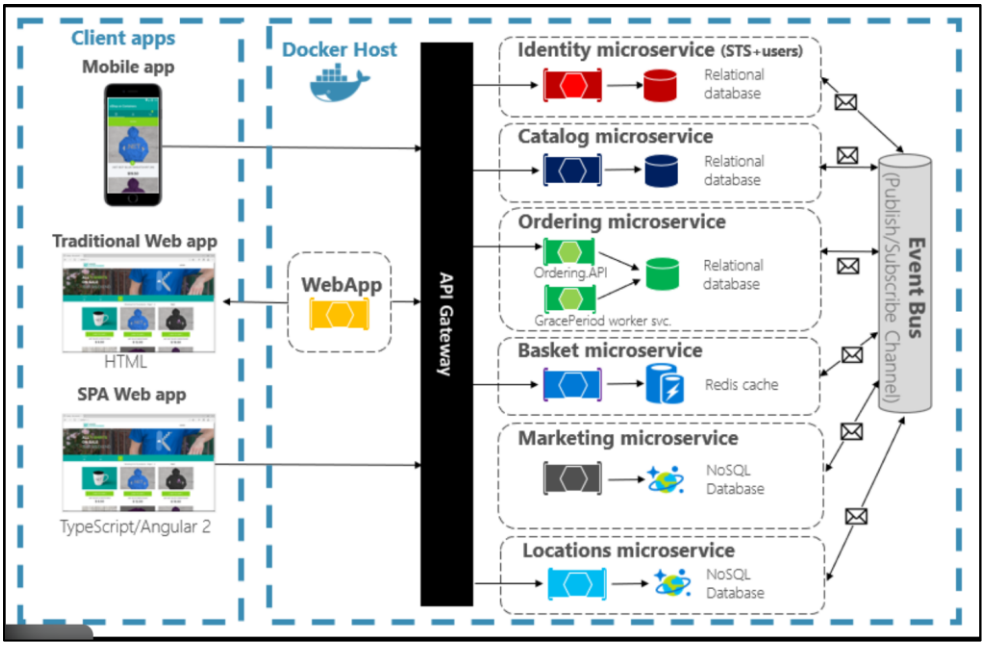
This architecture is suitable when:

- The FYP is deployed on the cloud
- The project requires scalability
- AI/ML or data analytics is involved
- The system is expected to grow over time

Diagram


AWS Cloud
Region
AZ1
Web Server
AZ2
Web Server
Load Balancer
Load Balancer
AZ1
App Server
AZ2
App Server
AZ1
Primary
Replication
AZ2
Secondary


Top 7 Cloud Native Architecture Patterns
Vishakha Sadhwani
@vsadhwani
Microservices Architecture
Client
Web
Mobile
PC
CDN
API Gateway
Identity Provider
Microservices
Service Registry & Discovery
Management
Domain A
Domain B
DB A
DB B
Database Sharding / Horizontal Scaling Architecture
Client Apps
Web
Mobile
API Gateway
Stateless Services
Microservices
Catalog
Shopping Cart
Discount
Ordering
Stateful DBs
MESSAGE BROKER
Even-driven Architecture
Event Broker
Event Consumer(s)
Event Producer
Event
Topic 01
Topic 02
Subscribe Topic 01
Subscribe Topic 02
Serverless Architecture
Client Apps
Web
Mobile
API
Data storage
Queue
Event Sources
Serverless Fuction
Side-Car Pattern
Ingress
Egress
Pod
Sidecar Proxy Container
Core Container
Sidecar Log Scraper Container
LOG AGGREGATOR
Shared File System
POD
API Gateway Pattern
Client Apps
Web
Mobile
API Gateway
Catalog
Shopping Cart
Discount
Ordering
DB
Strangler fig Pattern
Clients
Legacy Monolith
Strangler Facade


Client apps
Mobile app
Traditional Web app
HTML
SPA Web app
TypeScript/Angular 2
Docker Host
WebApp
API Gateway
Identity microservice (STS+users)
Relational database
Catalog microservice
Relational database
Ordering microservice
Ordering.API
GracePeriod worker svc.
Relational database
Basket microservice
Redis cache
Marketing microservice
NoSQL Database
Locations microservice
NoSQL Database
Event Bus
(Publish/Subscribe Channel)

Applying Data Architecture to an FYP (Step-by-Step)

Step 1: Understand Your FYP Requirements

- Type of data
- Data volume
- Real-time vs batch
- ML required or not

Step 2: Choose the Right Architecture

FYP Type	Recommended Architecture
Simple Web App	Monolithic / Layered
ML Project	Data Lake / Layered
Analytics	Data Warehouse
Real-Time System	Lambda / Kappa

Step 3: Map Architecture to Your Project

- Identify data sources
- Define storage
- Define processing
- Define outputs